«Общий знаменатель» — техдок для хакатона

Oct 25, 2026 · @Bodya

Обзор

На каждую идею заводится свой чат. Пока команда обсуждает, рядом сам собирается **канвас** — короткий тезисный конспект разговора в виде дерева. В конце обсуждения он превращается в визуальный тезис-план проекта, а его можно выгрузить в markdown или PNG.

Проблема: идею обсуждают в сотне сообщений. Через час никто не помнит, на чём сошлись, а новому человеку приходится читать всё с начала. На выходе остаётся чат, а не план.

Что делает агент:

- ведёт канвас: вытаскивает тезисы, раскладывает по блокам, сжимает повторы;
- различает **одну цель сказанную разными словами** (сводит в один тезис) и **настоящий конфликт** (помечает на канвасе и спрашивает в чате, что важнее);
- когда меняется ограничение, подсвечивает тезисы и шаги плана, которые придётся пересмотреть;
- по кнопке «Собрать план» строит тезис-план: этапы → шаги, каждый шаг привязан к тезисам, из которых он вырос.

В чат агент пишет редко: только на конфликт и на смену ограничения. Всё остальное видно на канвасе.

Подача: тема 3 «Второй мозг». Канвас накапливает контекст идеи, а агент применяет его в нужный момент: при конфликте, при смене требований, при сборке плана. Репозиторий открываем под MIT.

MVP (обязательно к 15:30):

1. Список идей → чат идеи на несколько участников.
2. Живой канвас: каркас блоков + дерево тезисов, обновляется по ходу разговора.
3. Сведение повторов и «одной цели» в один тезис; конфликты красным ребром + вопрос в чате.

4. Подсветка последствий при смене ограничения.
5. Кнопка «Собрать план» → режим тезис-плана + экспорт в markdown и PNG.

Что я бы добавил

Всё это дешёво в реализации и закрывает частые вопросы жюри.

Что	Зачем	Цена	Совет
Клик по тезису подсвечивает реплики, из которых он взят	Ответ на «а если ИИ наврал?»: каждый тезис проверяем	~30 мин фронта, `source_msgs` уже в модели	В MVP
Ручная правка и закрепление тезиса	Закреплённый тезис агент не переписывает	~30 мин + одна проверка в reducer	В MVP
Тестовые чаты с ожидаемым канвасом	Быстро проверить, что правка промпта ничего не сломала	Уже в доке, раздел «Тестовые чаты»	В MVP
«Догони меня»: что поменялось на канвасе с прошлого визита	Вау для демо: опоздавший за 10 секунд в курсе	~40 мин, дифф из журнала событий	Если останется время

Из прошлой версии убраны голосование и журнал решений. Их заменяют статусы тезисов на канвасе и итоговый план.

Канвас

Канвас = фиксированный каркас из 7 блоков, внутри каждого дерево тезисов глубиной 2. Каркас фиксирован, потому что так канвас всегда читается одинаково, вёрстка детерминированная, а модели нужно только выбрать блок и родителя. Свободная mind map красивее, но на живой модели расползается и прыгает на каждом обновлении.

Экран: слева список идей и чат идеи (этот фронт у вас уже есть), справа канвас. Между ними перетаскиваемый разделитель. Канвас по умолчанию занимает около 60% ширины.

Блоки

Блок	Что туда попадает	Пример тезиса
Суть	Одна фраза о том, что мы делаем. Переписывается Consolidator'ом	«Поднять продажи магазина на 15% без больших затрат»
Проблема	Боль, ради которой всё затевается	«Люди уходят до покупки»
Для кого	Аудитория и сегменты	«Старые клиенты, которые перестали покупать»
Решение	Предложения и функции; дети — способы и детали	«Короче путь до покупки» → «оформление в 2 шага»
Ограничения	Бюджет, сроки, люди, технические пределы	«2 разработчика, месяц»
Риски	Опасения и возражения	«Скидка съест маржу»
Открытые вопросы	Неразрешённое, в том числе вопросы по конфликтам	«Что важнее: вернуть клиентов или маржа?»

Уровень 1 — тезис (до 10 слов). Уровень 2 — деталь: способ, аргумент, пример. Глубже не идём: канвас — это конспект, а не стенограмма. На блок — не больше 6 тезисов. Если их стало больше, Consolidator сжимает.

Статусы тезиса

Статус	Когда	Как выглядит
`new`	Только что появился или изменился	Подсветка на 3 с, анимация появления
`active`	Обычное состояние	Карточка + аватары авторов; счётчик «+N», если поддержали другие
`disputed`	Участвует в неразрешённом конфликте	Красная рамка, красное ребро к оппоненту
`needs_review`	Изменилось ограничение, на котором тезис держался	Жёлтая рамка + пунктир к изменившемуся ограничению

Статус	Когда	Как выглядит
`rejected`	Конфликт разрешён не в его пользу	Серый, зачёркнут, причина в тултипе. Не удаляется, чтобы было видно, почему отказались
флаг `pinned`	Человек поправил тезис руками или закрепил	Булавка; агент не переписывает текст и не сливает

«Одна цель, разные слова» ребром не рисуем. Такие тезисы сводятся в один: сверху общая цель, дети — способы каждого автора и аватары обоих. Так сам канвас показывает, что люди хотели одного и того же.

Два режима

1. **Живой** (идёт обсуждение): блоки сеткой, тезисы появляются и перестраиваются с анимацией.
2. **План** (после «Собрать план»): правая панель переключается на план: сверху тезисы уровня 1 в одну строку на блок, ниже лента этапов со шагами. Навёл на шаг — подсвечиваются тезисы, из которых он вырос. Кнопки «Скачать .md» и «Скачать PNG». Чат при этом не закрывается: можно продолжить обсуждение и пересобрать план.

Как рисовать (для фуллстека)

- **Базовый вариант:** CSS grid из блоков-карточек, внутри — вложенный список. Один SVG-слой поверх для рёбер конфликтов и зависимостей (готовые стрелки — react-xarrows). Анимации через framer-motion с `layout`.
- **Если фуллстек уже работал с React Flow** — блоки как group nodes с фиксированными координатами, тезисы — дочерние узлы со стековой раскладкой. Бесплатно даёт pan/zoom.
- PNG: `html-to-image` (`toPng`) по DOM-узлу канваса. Markdown отдаёт бэкенд.

Сценарий демо

Идея «Поднять продажи интернет-магазина». Четверо играют роли с четырёх ноутбуков, на проекторе канвас. За 9 реплик жюри видит всё: сборку конспекта, сведение одной цели, конфликт, последствия и итоговый план. Сценарий лежит в `scenarios/demo_shop.json` и заодно является первым тестовым чатом.

#	Кто	Реплика	Что меняется на канвасе	Агент в чате
1	Руководитель	«Хотим поднять продажи на 15% за квартал, большого бюджета нет»	Суть: «+15% продаж за квартал». Ограничения: «бюджет небольшой»	—
2	Маркетолог	«Люди уходят без покупки, нужны персональные предложения»	Проблема: «уходят до покупки». Решение: «персональные предложения»	—
3	Дизайнер	«Давайте сократим путь до покупки, сейчас 5 шагов оформления»	**Одна цель.** Два тезиса сливаются в «Довести до покупки» с детьми «персональные предложения» [М] и «короче оформление» [Д]	Тост на канвасе: «Свёл 2 тезиса — цель одна»
4	Разработчик	«Сложные рекомендации не потянем, нас двое и месяц»	Ограничения: «2 разработчика, месяц». **Совместимые цели:** под «персональные предложения» появляется деталь агента «готовые наборы по простым правилам»	«Персональные предложения можно сделать готовыми наборами по простым правилам — без ML. Добавил на канвас как вариант»

#	Кто	Реплика	Что меняется на канвасе	Агент в чате
5	Маркетолог	«И нужна рассылка со скидкой 20% всем, чтобы вернуть старых клиентов»	Для кого: «старые клиенты». Решение: «рассылка со скидкой 20%». **Конфликт:** красное ребро с «бюджет небольшой», оба `disputed`. В «Открытых вопросах» появляется вопрос	«Тут настоящий конфликт: скидка 20% бьёт по марже, а бюджет ограничен. Что важнее: вернуть старых клиентов или сохранить маржу?»
6	Руководитель	«Маржа важнее. Скидка максимум 5% и только на наборы»	«Рассылка 20%» → `rejected` с причиной. Новая деталь «скидка 5% на наборы», зависит от бюджета. Вопрос закрыт	—
7	Руководитель	«Новости: бюджет дали, но срок — 2 недели вместо месяца»	Оба ограничения обновлены. **Последствия:** «скидка 5%» и «готовые наборы» желтеют (`needs_review`), пунктир к ограничениям	«Изменились бюджет и срок. Скидка 5% держалась на нехватке бюджета — можно пересмотреть. Наборы за 2 недели под вопросом: начать с оформления?»
8	Дизайнер	«Да, сначала оформление, наборы вторым этапом»	«Готовые наборы» снова `active` с пометкой «этап 2»	—

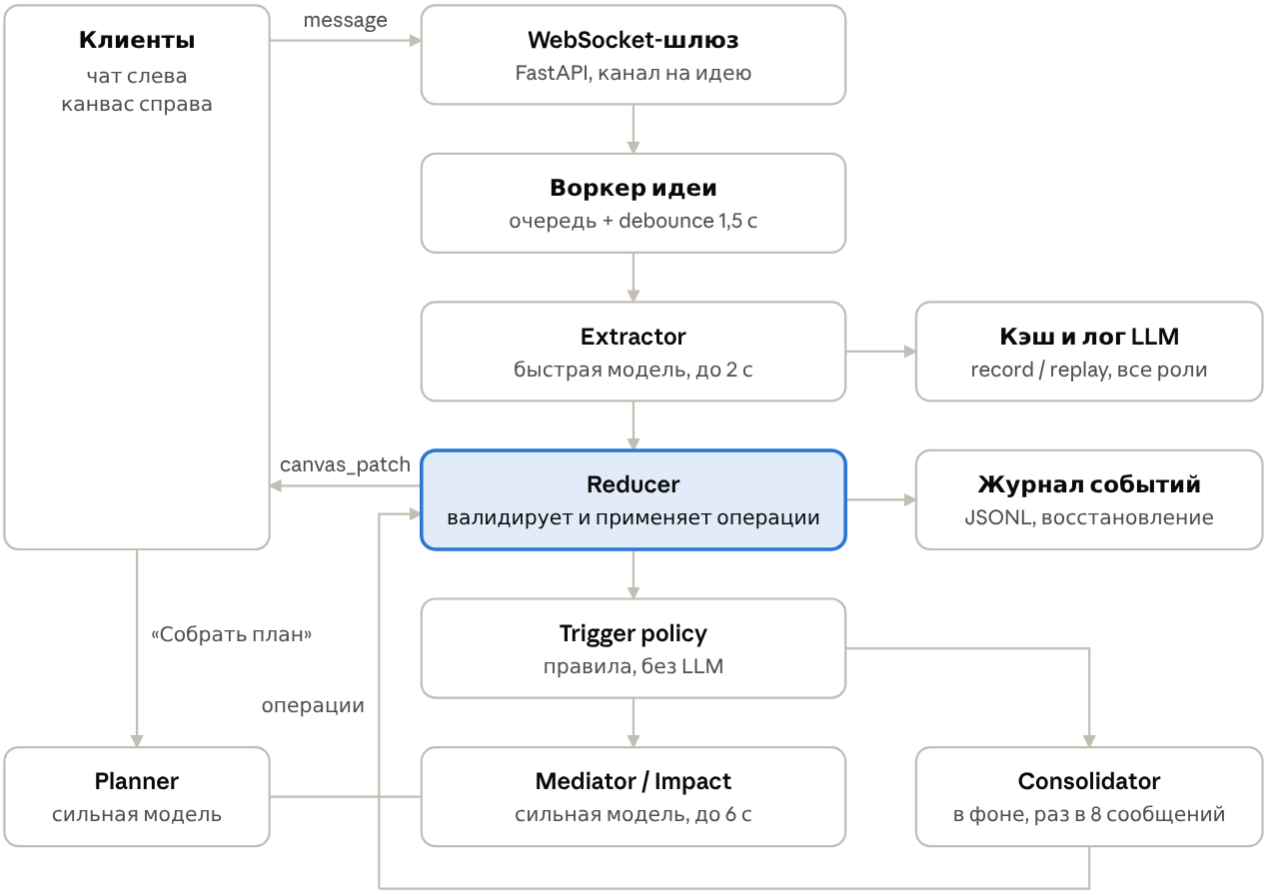
#	Кто	Реплика	Что меняется на канвасе	Агент в чате
9	Любой	Жмёт **«Собрать план»**	Режим плана: этап 1 «оформление в 2 шага», этап 2 «наборы + скидка на наборы», этап 3 «возврат старых клиентов» + оставшиеся вопросы. Навёл на шаг — подсветка исходных тезисов	«План собран, открытых вопросов: N»

Финал демо — нажать «Скачать .md» и показать one-pager. Если успеете «Догони меня», вставьте перед шагом 7 пятого участника, который «опоздал» и получает пересказ в три строки.

Архитектура

Один Python-сервис на FastAPI держит комнаты-идеи, состояние канваса и ML-пайплайн. Фронт только рисует. Главный принцип остаётся прежним: **LLM никогда не перегенерирует канвас целиком**. Она возвращает операции (добавить тезис, слить, перенести), а Reducer на Python проверяет и применяет их. Отсюда стабильные id узлов: анимации не дёргаются, закреплённые тезисы не теряются, ссылки тезис → сообщения живут.

LLM предлагает операции, состояние меняет только редьюсер



архитектура · путь от сообщения до канваса

Всё, что предлагают пять LLM-ролей, проходит через Reducer, и только он рассылает `canvas_patch` клиентам. Если триггера нет, цикл заканчивается после Extractor.

Чат на нашем бэке

Фронт чатов уже написан. Его подключаем к нашему бэку: список идей через REST, сообщения и патчи канваса через один WebSocket на комнату (протокол — в разделе «Бэкенд»). Источник правды для сообщений — бэк. Фронт не добавляет своё сообщение в ленту сам, а ждёт эхо `message` с выданным `seq`. Так у всех одинаковый порядок сообщений, а ссылки тезис → сообщение не ломаются.

Что доработать в готовом чате: подключение к WebSocket, отдельный стиль сообщений агента, подсветка сообщений по списку `seq` (для клика по тезису), индикатор «агент читает / думает».

Стек

Слой	Выбор	Почему
Бэкенд	Python 3.11+, FastAPI, uvicorn	Родной для ML, WebSocket из коробки
Схемы	Pydantic v2	Одни модели дают и JSON-схему для LLM, и валидацию
LLM-клиент	openai SDK с base_url на OpenRouter или провайдера	Модель меняется одной переменной
Хранение	Состояние в памяти + журнал событий JSONL на комнату	Ноль инфраструктуры; из журнала же строится «Догони меня»
Кэш LLM	Файловый, по хэшу запроса	Демо воспроизводимо и работает без сети
Канвас	React: CSS grid + react-xarrows + framer-motion (или React Flow)	См. раздел «Канвас»
Экспорт	markdown на бэке, PNG через html-to-image на фронте	Без headless-браузера на сервере

Пять LLM-ролей

Роль	Когда запускается	Модель	Что возвращает
Extractor	На каждую пачку сообщений (debounce 1,5 с)	Быстрая	Новые тезисы, привязки к существующим, смены ограничений, кандидаты в конфликт
Consolidator	Каждые 8 сообщений, или когда в блоке > 6 тезисов, в фоне	Сильная	Слияния, переносы, переформулировки, новая «Суть»
Mediator	Триггер конфликта или общей цели	Сильная	Вердикт same_goal / complementary / conflict / misunderstanding + операции + текст для чата
Impact	Изменилось ограничение, от которого что-то зависит	Сильная	Для каждого затронутого узла: keep / revise / drop + объяснение

Роль	Когда запускается	Модель	Что возвращает
Planner	Кнопка «Собрать план»	Сильная	Этапы → шаги со ссылками на тезисы, оставшиеся вопросы

«Догони меня», если будет, — шестая роль на быстрой модели: пересказ диффа канваса между двумя версиями.

Путь одного сообщения

1. Клиент шлёт `message` по WebSocket. Сервер выдаёт `seq`, рассылает сообщение всем и кладёт его в очередь комнаты.
2. Воркер комнаты ждёт 1,5 с и забирает пачку. По комнате всегда один обработчик.
3. Extractor → Reducer → всем уходит `canvas_patch`. Тезис появляется на канвасе примерно через 2–3 с после сообщения.
4. Trigger policy решает: нужен ли Mediator, Impact или Consolidator.
5. Если нужен, он отрабатывает, его операции снова проходят через Reducer. При необходимости в чат уходит сообщение агента.
6. Всё пишется в журнал событий и в лог LLM-вызовов.

Модель данных

Канвас хранится плоским списком узлов с `section` и `parent_id`, дерево строится при отрисовке. Id вида `N12`, `K3`, `Q2` выдаёт сервер, а не LLM. Модель ссылается только на уже существующие id или на временные `ref` внутри одного ответа.

```
# models.py
from typing import Literal
from pydantic import BaseModel

Section = Literal["essence", "problem", "audience", "solution",
                  "constraints", "risks", "questions"]
Status = Literal["new", "active", "disputed", "needs_review", "rejected"]

class Node(BaseModel):
    id: str                          # N12
    section: Section
    parent_id: str | None = None     # None = тезис уровня 1, иначе деталь
    text: str                        # до 10 слов
```

```
    goal: str | None = None          # зачем это предлагают (для поиска общей
цели)
    authors: list[str] = []          # кто высказал или поддержал; "agent" —
предложил агент
    source_msgs: list[int] = []      # seq сообщений, из которых взят
    status: Status = "new"
    status_reason: str | None = None # почему rejected / needs_review
    depends_on: list[str] = []       # id ограничений, на которых держится
    pinned: bool = False             # правлен человеком — агент не трогает
текст
    tag: str | None = None           # свободная пометка, например "этап 2"
    version: int = 1

class Conflict(BaseModel):
    id: str                          # K3
    a: str; b: str                   # id узлов
    explanation: str                 # за что конкурируют
    question_id: str                 # узел-вопрос в блоке questions
    status: Literal["open", "resolved"] = "open"
    resolution: str | None = None

class PlanStep(BaseModel):
    id: str; text: str
    from_nodes: list[str]            # из каких тезисов вырос шаг
    owner: str | None = None         # только если в чате кто-то взял на себя

class PlanStage(BaseModel):
    id: str; title: str; goal: str
    steps: list[PlanStep]

class Plan(BaseModel):
    stages: list[PlanStage]
    open_questions: list[str]        # id узлов-вопросов
    built_from_version: int          # с какой версией канваса собран

class Canvas(BaseModel):
    version: int = 0
    nodes: list[Node] = []
    conflicts: list[Conflict] = []
    plan: Plan | None = None
    mode: Literal["live", "plan"] = "live"

class Message(BaseModel):
    seq: int; author: str; role: str; text: str; ts: float
    is_agent: bool = False
```

```
class Idea(BaseModel):                # одна комната = одна идея
    id: str; title: str; created_at: float
    participants: dict[str, str] = {}   # user -> role
    messages: list[Message] = []
    canvas: Canvas = Canvas()
    counters: dict[str, int] = {}
```

Узел «Суть» всегда один, его текст меняет только `set_essence`. Ограничения — обычные узлы в блоке `constraints`: на них ссылаются `depends_on` других узлов.

Операции

Закрытый набор. Их возвращают LLM и UI, применяет Reducer. Операция с несуществующим id или нарушающая правила (правка pinned, глубина > 2) отбрасывается и логируется, остальные применяются.

Операция	Поля	Кто вызывает
`add_node`	section, parent_id или parent_ref, text, goal, source_msg, possible_conflict_with	Extractor, Mediator
`attach_source`	node_id, source_msg, author — повтор или поддержка	Extractor
`update_node`	node_id, text	Extractor, Consolidator (кроме pinned)
`change_constraint`	node_id, new_text	Extractor
`resolve_question`	question_id, answer, source_msg	Extractor
`merge_nodes`	node_ids, new_text, goal — старые становятся детьми нового или сливаются полностью	Consolidator, Mediator
`move_node`	node_id, section, parent_id	Consolidator
`set_essence`	text	Consolidator
`add_conflict`	a, b, explanation, question_text, options	Mediator
`resolve_conflict`	conflict_id, keep_ids, reject_ids, reason	Mediator
`set_status`	node_id, status, reason	Mediator, Impact
`add_dependency`	node_id, constraint_id	Extractor, Mediator

Операция	Поля	Кто вызывает
`set_plan`	plan	Planner
`edit_node` / `pin` / `unpin`	node_id, text	UI (человек), без LLM

Журнал `data/{idea_id}.jsonl` хранит сообщения и применённые операции с версией канваса. При рестарте состояние собирается проигрыванием журнала без LLM. Из него же считается дифф для «Догони меня».

ML-пайплайн, шаг 1: Extractor и Consolidator

Конспект строится в два контура. Быстрый контур (Extractor) на каждой пачке сообщений добавляет тезисы, чтобы канвас жил. Медленный контур (Consolidator) раз в несколько сообщений наводит порядок: сливает, переносит, переформулирует. Так канвас и быстрый, и не расползается.

Extractor (быстрая модель, до 2 с)

Вход: новые сообщения, 8 предыдущих для контекста, компактный канвас: по блокам, одна строка на узел — `N4 [solution] Довести до покупки | цель: конверсия | авторы: М, Д`.

Выход: тип каждого сообщения (idea, requirement, objection, change, agreement, question, answer, chatter) и операции `add_node`, `attach_source`, `update_node`, `change_constraint`, `resolve_question`, `add_dependency`.

Правила, которые делают конспект конспектом, а не стенограммой:

1. **Тезис — до 10 слов, без воды.** «Ну я думаю, что нам бы не помешало…» → «Персональные предложения».
2. **Повтор не создаёт узел.** То же самое другими словами или «согласен с ним» → `attach_source`, счётчик «+1» на существующем узле.
3. **Деталь идёт под свой тезис.** Уточнение, способ, пример → `add_node` с `parent_id`.
4. **Цель у каждого предложения.** Для узлов в `solution` заполняется `goal` — зачем это предлагают, 2–5 слов. Модель обязана переиспользовать формулировку цели, которая уже есть на канвасе, если смысл тот же. Это **канонизация целей**: два узла разных авторов с одинаковым `goal` — кандидат в «одну цель», и находится он одним groupby в Python без лишнего вызова LLM.

5. **Смена ограничения — не новый узел.** «Бюджет дали» → `change_constraint` на существующий узел. Это запускает impact-анализ.
6. **Зависимости.** Если предложение явно опирается на ограничение («раз денег мало, то…»), — `add_dependency`.
7. **Конфликт не решает.** Только помечает `possible_conflict_with`. Вердикт выносит Mediator на сильной модели.
8. **Ничего от себя.** Только то, что сказали люди. Все добавления от агента делает Mediator, и они помечены автором «agent».

Consolidator (сильная модель, в фоне)

Когда: каждые 8 сообщений людей; или в блоке стало больше 6 тезисов; или перед сборкой плана. Не запускается, пока работает Mediator.

Вход: весь канвас (с авторами, статусами, pinned) и последние 20 сообщений.

Выход: `merge_nodes`, `move_node`, `update_node`, `set_essence`. Максимум 6 операций за проход, чтобы канвас не «перетряхивало» на глазах у людей.

Что исправляет: дубли, которые пропустил Extractor; тезисы не в том блоке; детали, которые стали самостоятельными тезисами, и наоборот; слишком длинные формулировки; «Суть» — одна фраза про текущее состояние идеи.

Что ему запрещено (проверяет Reducer): трогать pinned-узлы; сливать узлы из открытого конфликта (это и есть принудительное слияние); удалять узлы — операции удаления в наборе нет. Слитый узел забирает все `source_msgs`, `authors` и `depends_on`, так что ничей вклад не теряется.

Reducer: порядок после Extractor

1. Валидация JSON по Pydantic. При ошибке один ретрай с текстом ошибки, потом пропуск с записью в лог.
2. Отброс операций с несуществующими id, глубиной > 2 и правками pinned.
3. Выдача id новым узлам, разрешение `parent_ref`.
4. Сигналы для trigger policy: группы по `goal`, флаги конфликтов, смены ограничений, ответы на вопросы, счётчик сообщений для Consolidator.
5. Статус `new` переходит в `active` при следующей версии канваса. Подсветку на 3 с рисует фронт.

ML-пайплайн, шаг 2: Mediator — одна цель или конфликт

Когда агенту вмешиваться, решают правила на Python. Что именно сделать с канвасом и написать в чат, решает сильная модель. На канвасе агент действует сам, в чат пишет только тогда, когда нужен ответ людей.

Trigger policy

Триггер	Условие	Кто отрабатывает	Приоритет
`IMPACT`	`change_constraint`, и от этого ограничения зависит хотя бы один узел или шаг плана	Impact	1
`RESOLVE`	`resolve_question` по вопросу, за которым стоит открытый конфликт	Mediator	2
`CONFLICT`	У нового узла есть `possible_conflict_with`	Mediator	2
`SAME_GOAL`	≥ 2 активных тезиса разных авторов с одним `goal`, ещё не сведены	Mediator	3
`CONSOLIDATE`	8 сообщений с прошлого прохода, или блок > 6 тезисов	Consolidator	4

За один цикл срабатывает один триггер с наивысшим приоритетом, остальные ждут следующего. Один и тот же набор узлов дважды в Mediator не попадает: отработанные пары запоминаем. Пороги вынесены в конфиг.

Четыре вердикта

Вердикт	Признак	Что происходит на канвасе	Чат
`same_goal`	Одна цель, разные средства	`merge_nodes`: общий тезис-цель, способы авторов — дети	Молчит, тост на канвасе «Свёл N тезиса»

Вердикт	Признак	Что происходит на канвасе	Чат
`complementary`	Разные цели, но совместимые	`add_node` деталь-компромисс от автора «agent»	Одна фраза: что предложил и чьи цели оно учитывает
`conflict`	Цели конкурируют за один ресурс, одновременно недостижимы	`add_conflict`: оба узла `disputed`, красное ребро, вопрос в «Открытых вопросах»	Называет конфликт, 2–3 варианта с ценой каждого, спрашивает, что важнее
`misunderstanding`	Спорят, но говорят о разном или одном и том же	Узел-вопрос в «Открытых вопросах»	Уточняющий вопрос авторам

По `RESOLVE` Mediator читает ответ людей и делает `resolve_conflict`. Проигравший узел становится `rejected` с причиной, выигравший снова `active`. Если ответ содержит новый компромисс («скидка 5% только на наборы»), он попадает на канвас новым узлом с `depends_on`.

Порядок рассуждения

Порядок полей в JSON-схеме задаёт порядок мышления модели. Вердикт идёт последним:

1. `goals` — что на самом деле хочет каждый автор;
2. `same_underlying_goal` + почему;
3. `jointly_satisfiable` — можно ли удовлетворить всех сразу при текущих ограничениях; если нет — `contested_resource` (деньги, время, маржа, люди, внимание пользователя);
4. `verdict` и `confidence`;
5. `ops` и `chat_message`.

Защита от принудительного слияния (guards в коде)

- `jointly_satisfiable = false` → `merge_nodes` и деталь-компромисс запрещены, разрешён только `add_conflict`. Нарушение → ретрай с замечанием.
- `confidence < 0.6` → вместо вердикта уточняющий вопрос.

- При `merge_nodes` каждый исходный узел должен остаться ребёнком нового или передать ему своих авторов. Молча потерять чьё-то предложение нельзя.
- Pinned-узлы не сливаются и не переписываются. Если pinned-узел участвует в same_goal, он становится родителем, а остальные уходят к нему детьми.

На питче: «Агент физически не может склеить конфликт и потерять чьё-то предложение — это гарантирует код, а не промпт».

Стиль сообщений агента

До 3 предложений: что заметил → что сделал на канвасе или что спрашивает. К людям обращается по ролям, без комплиментов и канцелярита. У сообщения есть `focus_ids`: клик по нему подсвечивает нужные узлы на канвасе.

Последствия и итоговый тезис-план

Impact-анализ

Подсветка детерминированная, LLM только объясняет. Поэтому на демо она сработает надёжно. Зависимости берутся из `depends_on` узлов и из `from_nodes` шагов плана.

```
# impact.py
def affected(canvas: Canvas, changed: set[str]) -> tuple[list[Node],
list[PlanStep]]:
    hit, frontier = set(), set(changed)
    while frontier:                                  # узлы могут зависеть
от узлов
        nxt = {n.id for n in canvas.nodes
               if n.id not in hit and n.status != "rejected"
               and frontier & set(n.depends_on)}
        hit |= nxt
        frontier = nxt
    nodes = [n for n in canvas.nodes if n.id in hit]
    steps = [s for st in (canvas.plan.stages if canvas.plan else []) for s in
st.steps
             if set(s.from_nodes) & (hit | changed)]
    return nodes, steps
```

1. Найденные узлы сразу получают `needs_review`, шаги плана — флаг `stale`. Канвас желтеет ещё до ответа LLM.

2. Impact-промпт получает старое и новое ограничение и затронутые узлы. Для каждого узла возвращает `keep` / `revise` / `drop`, причину и для `revise` новую формулировку. И одно сообщение для чата.
3. `keep` → узел снова `active`. `revise` → узел остаётся `needs_review`, а предложенная формулировка показывается под ним. `drop` → вопрос в «Открытых вопросах». Статус `needs_review` снимается, когда Extractor видит ответ людей по этому узлу, или человек нажимает «Ок» на карточке.
4. Если план уже собран, над ним появляется плашка «План устарел: N шагов зависят от изменившегося» с кнопкой «Пересобрать».

Planner: «Собрать план»

По кнопке сначала идёт проход Consolidator, потом Planner на сильной модели. На входе весь канвас без `rejected`-узлов, на выходе `Plan`.

Правила для Planner (часть проверяет код):

- 2–4 этапа, в каждом 2–5 шагов. Шаг — действие, начинается с глагола, до 12 слов.
- **Каждый шаг ссылается хотя бы на один существующий узел** (`from_nodes`). Шаги без ссылки код отбрасывает: план не должен содержать того, что не обсуждали.
- Порядок этапов учитывает ограничения, пометки `tag` («этап 2») и договорённости из чата.
- Узлы `disputed` и `needs_review` в шаги не превращаются. Они идут в `open_questions` вместе с открытыми вопросами.
- Ответственный (`owner`) указывается, только если в чате кто-то явно взял задачу на себя.

План живёт в `canvas.plan` вместе с `built_from_version`. Пересборка заменяет его целиком: это единственное место, где LLM генерирует структуру заново, и это нормально: план — снимок, а не живая структура.

Экспорт

- **Markdown** — `GET /ideas/{id}/export.md`, собирается шаблоном на бэке без LLM:

```
# Поднять продажи интернет-магазина
> Суть: +15% продаж за квартал без больших затрат

## Проблема
- Люди уходят до покупки
## Решение
- Довести до покупки
  - Оформление в 2 шага
  - Готовые наборы по простым правилам
## Отказались
- ~~Рассылка со скидкой 20%~~ — маржа важнее
## План
### Этап 1. Оформление
1. Сократить оформление до 2 шагов
## Открытые вопросы
- ...
```

- **PNG** — на фронте через `html-to-image` по DOM-узлу правой панели (канвас или план — что открыто). Перед снимком фронт разворачивает все блоки и скрывает кнопки.

Промпты

Промпты лежат файлами в `prompts/` под git: так видно, какая правка что сломала в тестовых чатах. Ответ везде через structured output (JSON schema из Pydantic), температура 0–0,2. Во все промпты канвас передаётся одним и тем же компактным рендером (`render.canvas_compact`):

```
[essence] N1 +15% продаж за квартал
[problem] N3 Люди уходят до покупки | авторы: Маркетолог
[solution] N4 Довести до покупки | цель: конверсия | авторы: Маркетолог,
Дизайнер
  N5 Персональные предложения | авторы: Маркетолог
  N6 Оформление короче 5 шагов | авторы: Дизайнер | PINNED
[constraints] N2 Бюджет небольшой
[questions] Q1 Что важнее: вернуть клиентов или маржа? | конфликт K1: N7 vs
N2
```

Extractor

```
Ты ведёшь краткий тезисный конспект командного обсуждения идеи — канвас.
Канвас состоит из блоков: essence (суть), problem, audience, solution,
constraints, risks, questions. В блоке — тезисы (уровень 1) и детали под ними
(уровень 2). Глубже нельзя.
```

```
Разбери НОВЫЕ сообщения и верни операции над канвасом. Людям не отвечай,
решений не предлагай — только конспектируй сказанное.

Правила:
1. Тип каждого сообщения: idea, requirement, objection, change, agreement,
question,
   answer, chatter. На chatter операций нет.
2. Тезис — до 10 слов, без вводных и оценок, на языке чата.
3. Если сообщение повторяет или поддерживает уже существующий узел —
attach_source,
   НЕ новый узел.
4. Уточнение, способ, пример к существующему тезису — add_node с parent_id
этого тезиса.
5. Для каждого узла в solution укажи goal — ЗАЧЕМ это предлагают, 2-5 слов.
   Если по смыслу это та же цель, что у другого узла, скопируй её
формулировку
   дословно. Разные способы — не повод для разных целей.
6. Если меняется или снимается ограничение (бюджет, срок, люди) —
change_constraint
   на СУЩЕСТВУЮЩИЙ узел в constraints.
7. Если предложение явно опирается на ограничение — add_dependency.
8. Если сообщение отвечает на вопрос из questions — resolve_question.
9. Если новый узел ПОХОЖЕ противоречит другому узлу или ограничению, укажи их
   в possible_conflict_with. Не решай, настоящий ли это конфликт.
10. Узлы с пометкой PINNED не меняй.
11. Ссылайся только на id из канваса. Новым узлам давай временный ref ("r1"),
    чтобы сослаться на них как на родителя в том же ответе.
12. Не добавляй того, чего люди не говорили.
```

Пользовательское сообщение: `КАНВАС:` + компактный рендер, `КОНТЕКСТ:` + 8 предыдущих сообщений, `НОВЫЕ СООБЩЕНИЯ:` в формате `#14 [Разработчик] текст`.

```
class ExtractOp(BaseModel):
    op: Literal["add_node", "attach_source", "update_node",
"change_constraint",
                "resolve_question", "add_dependency"]
    ref: str | None = None              # временный id нового узла
    section: Section | None = None
    parent_id: str | None = None        # существующий id или ref
    node_id: str | None = None          # для
attach/update/change/resolve/dependency
    text: str | None = None
    goal: str | None = None
    constraint_id: str | None = None    # для add_dependency
    possible_conflict_with: list[str] = []
    source_msg: int
    author: str

class ExtractResult(BaseModel):
    labels: list[tuple[int, Literal["idea", "requirement", "objection",
"change",
                                    "agreement", "question", "answer",
"chatter"]]]
    ops: list[ExtractOp]
```

Consolidator

```
Ты редактор конспекта-канваса. Канвас собирался на ходу и мог расползтись.
Наведи порядок, НЕ меняя смысла сказанного:
- слей дубли (merge_nodes);
- перенеси узлы, стоящие не в своём блоке, или не под своим родителем
(move_node);
- сократи длинные и размытые формулировки (update_node);
- обнови суть — одна фраза до 15 слов о том, что команда сейчас собирается
  сделать (set_essence).
В блоке должно остаться не больше 6 тезисов.
Нельзя: трогать PINNED; сливать узлы из открытого конфликта; выбрасывать чьи-
то
предложения; добавлять новое содержание. Не больше 6 операций. Если канвас
в порядке, верни пустой список.
```

Mediator

```
Ты фасилитатор командного обсуждения. Система заметила ситуацию {trigger}
вокруг узлов {focus_ids}. Разберись, что происходит, и не объединяй насильно:

- same_goal: люди хотят одного, предлагают разные способы → merge_nodes:
общий
  тезис-цель, способы — его дети. В чат не пиши.
- complementary: цели разные, но достижимы вместе → add_node: деталь-
компромисс,
  выполнимый при ограничениях. В чат — одна фраза: что добавил и чьи цели
учёл.
- conflict: цели конкурируют за ресурс (деньги, время, маржу, людей, внимание
  пользователя), выигрыш одной стороны — проигрыш другой → add_conflict. НЕ
предлагай
  решение. В чат: назови конфликт, дай 2-3 варианта приоритета и чем жертвует
  каждый, спроси, что важнее.
- misunderstanding: спорят, но говорят о разном или одном и том же →
уточняющий вопрос.

Если trigger = RESOLVE: люди ответили на вопрос конфликта {conflict_id}.
Сделай
resolve_conflict: какие узлы остаются, какие отклонены и почему (словами
людей). Если
в ответе есть новый вариант — add_node с depends_on.

Рассуждай по полям ответа в их порядке: что хочет каждый → одна ли цель →
можно ли удовлетворить всех сразу при ограничениях → вердикт → операции →
чат.
Если сомневаешься (confidence < 0.6) — спроси, а не утверждай.
Чат: до 3 предложений, обращайся к людям по ролям, без комплиментов.
PINNED не меняй. Ссылайся только на существующие id.
```

```
class MediatorResult(BaseModel):
    goals: list[tuple[str, str]]          # (node_id, что автор на самом деле
хочет)
    same_underlying_goal: bool
    same_goal_reason: str
    jointly_satisfiable: bool
    contested_resource: str | None
    verdict: Literal["same_goal", "complementary", "conflict",
"misunderstanding"]
    confidence: float
    ops: list[CanvasOp]                    # merge_nodes / add_node /
add_conflict / resolve_conflict
    chat_message: str | None               # None = агент в чат не пишет
    toast: str | None                      # короткая плашка на канвасе
```

Impact

```
У команды изменилось ограничение. Было: "{old}". Стало: "{new}".
Ниже узлы канваса и шаги плана, которые на нём держались.
Для каждого узла верни: keep (всё ещё в силе, почему) / revise (нужно
изменить, новая формулировка до 10 слов) / drop (потерял смысл, вопрос
команде).
Затем одно сообщение для чата, до 4 предложений: что изменилось и что
придётся пересмотреть. Не трогай узлы, которых нет в списке.
```

Planner

```
Собери тезис-план проекта по канвасу обсуждения.
- 2–4 этапа, у каждого название и цель в одну строку; в этапе 2–5 шагов.
- Шаг — действие с глагола, до 12 слов.
- Каждый шаг ОБЯЗАН ссылаться на узлы канваса, из которых вырос (from_nodes).
  Не добавляй шаги про то, что не обсуждалось.
- Порядок этапов — исходя из ограничений, пометок и договорённостей в чате.
- Узлы DISPUTED и NEEDS_REVIEW и открытые вопросы не превращай в шаги —
  перечисли их в open_questions.
- owner указывай, только если в чате кто-то явно взял задачу на себя.
```

Бэкенд

Один FastAPI-процесс: REST для списка идей и служебных действий, один WebSocket на идею для чата и канваса. **Первым делом, до 11:45, фиксируешь с фуллстеком протокол ниже и поднимаешь mock-режим**: бэк без LLM отдаёт заготовленные патчи из `demo_shop.json`. Тогда фронт не ждёт ML.

REST

Метод	Путь	Что делает
GET	`/ideas`	Список идей: id, title, суть, число сообщений, дата
POST	`/ideas`	Создать идею `{title}` → `{id}`; канвас с пустыми блоками
GET	`/ideas/{id}`	Полное состояние: сообщения + канвас + план
POST	`/ideas/{id}/plan`	Собрать или пересобрать план (асинхронно, результат придёт по WS)
GET	`/ideas/{id}/export.md`	Markdown one-pager
GET	`/ideas/{id}/digest?since=<version>`	«Догони меня»: пересказ изменений канваса (если будет)
POST	`/ideas/{id}/replay`	Проиграть сценарий из `scenarios/*.json` с паузами
POST	`/ideas/{id}/reset`	Очистить идею
GET	`/debug/llm`	Последние LLM-вызовы: роль, промпт, ответ, задержка, токены

WebSocket `/ws/ideas/{id}?user=Аня&role=маркетолог`

Направление	type	Payload
клиент → сервер	`message`	`{text, client_id}` — `client_id` для сопоставления с эхо
клиент → сервер	`edit_node`	`{node_id, text}` → узел становится pinned
клиент → сервер	`pin` / `unpin`	`{node_id}`
клиент → сервер	`ack_review`	`{node_id}` — «Ок» на жёлтой карточке
клиент → сервер	`set_mode`	`{mode: live/plan}` — если плана нет, запускает Planner
сервер → клиент	`snapshot`	`{messages, canvas}` — сразу после подключения

Направление	type	Payload
сервер → клиент	`message`	`{seq, author, role, text, ts, is_agent, client_id?, focus_ids?}`
сервер → клиент	`canvas_patch`	`{version, ops: [...применённые операции с итоговыми узлами]}`
сервер → клиент	`plan`	`{plan, stale: bool}`
сервер → клиент	`toast`	`{text, focus_ids}` — «Свёл 2 тезиса»
сервер → клиент	`agent_status`	`{status: idle/reading/thinking/planning}`

В `canvas_patch` сервер шлёт не сырые операции LLM, а результат: `upsert_node` (узел целиком), `remove_node` (только после merge), `upsert_conflict`. Фронту не нужно знать логику редьюсера. Правило: если `version` не равен своей + 1, фронт перезапрашивает `GET /ideas/{id}`.

Структура репозитория

```
backend/
  app/
    main.py          # FastAPI, REST, WebSocket
    ideas.py         # Idea: состояние, очередь, воркер, broadcast
    models.py        # Pydantic: канвас, операции, ответы LLM
    reducer.py       # применение и валидация операций, выдача id
    guards.py        # запрет принудительного слияния, pinned, глубина
    triggers.py      # trigger policy
    impact.py        # граф зависимостей
    llm.py           # клиент, structured output, ретраи, кэш, лог
    pipeline.py      # extract -> reduce -> trigger ->
mediate/impact/consolidate
    planner.py       # сборка плана + валидация from_nodes
    render.py        # канвас -> текст для промпта; канвас -> markdown
  prompts/           # extractor.md, consolidator.md, mediator.md, impact.md,
planner.md
  scenarios/         # demo_shop.json, teammates.json, coffee.json
  data/              # jsonl-журналы, llm_cache/, llm_log.jsonl
frontend/            # готовый чат + панель канваса
```

Скелет пайплайна

```
# ideas.py — один воркер на идею, гонок нет
class IdeaRoom:
    def __init__(self, idea: Idea):
        self.idea, self.sockets = idea, {}
        self.inbox: asyncio.Queue = asyncio.Queue()
        self.worker = asyncio.create_task(self._run())

    async def on_message(self, author, role, text, client_id):
        msg = Message(seq=len(self.idea.messages), author=author, role=role,
                      text=text, ts=time.time())
        self.idea.messages.append(msg)
        journal.append(self.idea.id, "message", msg)
        await self.broadcast({"type": "message", **msg.model_dump(),
"client_id": client_id})
        await self.inbox.put(("messages", msg))

    async def _run(self):
        while True:
            kind, first = await self.inbox.get()
            if kind == "plan":
                await safe(build_plan(self)); continue
            batch = [first]
            await asyncio.sleep(cfg.DEBOUNCE_S)
            while not self.inbox.empty():
                k, m = self.inbox.get_nowait()
                if k == "messages": batch.append(m)
                else: self.inbox.put_nowait((k, m)); break
            await safe(process_batch(self, batch))
```

```
# pipeline.py
async def process_batch(room: IdeaRoom, batch: list[Message]):
    cv = room.idea.canvas
    await room.status("reading")
    res = await llm.structured(cfg.FAST_MODEL, prompts.extractor,
                               render.extractor_input(cv, room.idea.messages,
batch), ExtractResult)
    patch, signals = reducer.apply_extract(cv, res)
    await room.patch(patch)

    trig = triggers.pick(cv, signals, room.idea.messages)
    if trig is None:
        return await room.status("idle")
    await room.status("thinking")
```

```
    if trig.kind == "IMPACT":
        nodes, steps = impact.affected(cv, trig.changed_ids)
        await room.patch(reducer.mark_needs_review(cv, nodes, steps,
trig.reason))
        res = await llm.structured(cfg.STRONG_MODEL, prompts.impact,
                                   render.impact_input(cv, trig, nodes,
steps), ImpactResult)
        patch = reducer.apply_impact(cv, res)
    elif trig.kind == "CONSOLIDATE":
        res = await llm.structured(cfg.STRONG_MODEL, prompts.consolidator,
                                   render.consolidator_input(cv,
room.idea.messages), ConsolidateResult,
                                   validator=guards.check_consolidate(cv))
        patch = reducer.apply_consolidate(cv, res)
    else:  # CONFLICT / SAME_GOAL / RESOLVE
        res = await llm.structured(cfg.STRONG_MODEL,
prompts.mediator.format(**trig.prompt_vars()),
                                   render.mediator_input(cv,
room.idea.messages), MediatorResult,
                                   validator=guards.check_mediator(cv, trig))
        patch = reducer.apply_mediator(cv, res, trig)

    await room.patch(patch)
    if getattr(res, "chat_message", None):
        await room.agent_say(res.chat_message, focus_ids=trig.focus_ids)
    if getattr(res, "toast", None):
        await room.broadcast({"type": "toast", "text": res.toast,
"focus_ids": trig.focus_ids})
    triggers.mark_done(cv, trig)
    await room.status("idle")
```

`guards.check_*` возвращают список нарушений. Если он не пустой, `llm.structured` делает один ретрай и дописывает нарушения в диалог. `safe()` ловит исключения, пишет их в лог и ставит статус `idle` — демо не падает из-за одного плохого ответа.

Модели, задержка и устойчивость

Две модели: быстрая для извлечения и сильная для вмешательств. Обе меняются одной переменной окружения. В 11:00 прогоните один и тот же запрос на 2–3 кандидатах и зафиксируйте выбор по задержке и валидности JSON.

Роль	Класс модели	Примеры	Бюджет задержки
Extractor, «Догони меня»	Быстрая, дешёвая, умеет structured output	Claude Haiku 5.5, модели класса mini/flash через OpenRouter	до 2 с
Mediator, Impact	Сильная в рассуждении	Claude Sonnet 5.5, модели класса pro	до 6 с
Consolidator	Сильная, работает в фоне	Та же, что Mediator	до 8 с, люди не ждут
Planner	Сильная	Та же, что Mediator	до 10 с, пока идёт анимация «Собираю план…»
Эмбеддинги (после MVP)	Multilingual	multilingual-e5 локально или embeddings API — подсказки дублей для Extractor	до 0,3 с

Если доступен только GigaChat или YandexGPT, заранее проверьте, поддерживают ли они structured output или function calling. Если нет — схема в тексте промпта, извлечение JSON из ответа и ретрай при ошибке валидации.

llm.py: что в нём должно быть

```
# llm.py
import hashlib, json, time
from openai import AsyncOpenAI

client = AsyncOpenAI(base_url=cfg.LLM_BASE_URL, api_key=cfg.LLM_API_KEY)

async def structured(model, system, user, schema, validator=None,
max_retries=1):
    key = hashlib.sha256(f"{model}|{system}|{user}".encode()).hexdigest()
    if cfg.CACHE_MODE in ("replay", "record") and (hit := cache.get(key)):
        return schema.model_validate_json(hit)      # мгновенно и без сети

    messages = [{"role": "system", "content": system},
                {"role": "user", "content": user}]
    for attempt in range(max_retries + 1):
        t0 = time.perf_counter()
```

```
        resp = await client.chat.completions.create(
            model=model, messages=messages, temperature=0.1,
timeout=cfg.LLM_TIMEOUT,
            response_format={"type": "json_schema",
                             "json_schema": {"name": schema.__name__,
                                             "schema":
schema.model_json_schema()}})
        raw = resp.choices[0].message.content
        log_call(model, messages, raw, time.perf_counter() - t0, resp.usage,
attempt)
        try:
            result = schema.model_validate_json(raw)
            problems = validator(result) if validator else []
            if not problems:
                if cfg.CACHE_MODE == "record":
                    cache.put(key, raw)
                return result
        except ValidationError as e:
            problems = [str(e)]
        messages += [{"role": "assistant", "content": raw},
                     {"role": "user", "content": "Исправь ответ: " + ";
".join(problems)}]
    raise LLMFailed(problems)
```

Точные параметры `response_format` зависят от провайдера: сверьтесь с его документацией в первые 15 минут. Некоторые модели надёжнее работают через tool calling с одним инструментом.

Три режима кэша — главная MLOps-страховка

- `off` — всегда живые вызовы (разработка, тестовые чаты).
- `record` — живые вызовы, удачные ответы пишутся в кэш. Включаете на финальной репетиции демо.
- `replay` — сначала кэш, при промахе живой вызов. На защите сценарий, отрепетированный слово в слово, отрабатывает мгновенно и так же, как на репетиции. А если жюри попросит написать своё сообщение, оно уйдёт в живую модель.

Ключ кэша — хэш всего промпта вместе с состоянием. Поэтому тексты сообщений на защите должны совпадать с репетицией до символа. Решение: у каждого игрока есть кнопка «вставить следующую реплику» (скрытая демо-панель, см. раздел рисков). Debounce на защите поставьте таким же, как на репетиции, и шлите реплики с паузой больше debounce, иначе пачки соберутся по-другому.

Деградация при отказах

- Таймаут или ошибка Extractor → сообщение остаётся в чате без карточек. Следующая пачка захватит его в контекст.
- Ошибка Mediator → агент молчит, триггер не помечается как отработанный и сработает ещё раз.
- Основной провайдер лежит → запасной ключ или модель в конфиге (`FALLBACK_MODEL`).

Наблюдаемость

Каждый вызов пишется в `data/llm_log.jsonl`: модель, промпт, ответ, задержка, токены, номер попытки, версия промпта (git hash). Страница `/debug/llm` показывает последние 20 вызовов. Если пара минут найдётся, можно подключить Langfuse. Но и jsonl хватит, чтобы показать жюри пайплайн вживую.

Тестовые чаты

Три чата для проверки: демо-сценарий с интернет-магазином (выше) и два новых. Каждый лежит в `scenarios/*.json` и проигрывается через `POST /ideas/{id}/replay`. После каждой правки промпта прогоняем все три и сверяем канвас с ожиданием. Текст тезисов может отличаться, важны структура и поведение агента.

Формат файла:

```
{"id": "teammates", "title": "Поиск сокомандников",
 "messages": [{"author": "Аня", "role": "продакт", "text": "..."}],
 "actions": [{"after": 10, "do": "build_plan"}]}
```

Чат 2. Поиск сокомандников на хакатоны

Проверяет всё сразу: одну цель, шум, конфликт с разрешением, смену ограничения, план.

```
#1  [Аня, продакт]      Идея: приложение, где студенты находят сокомандников
на хакатоны. Сейчас все ищут в чатах, это ад
#2  [Миша, разработчик] Сделаем анкеты со стеком и мэтчинг по навыкам
#3  [Лера, дизайнер]   Я бы сделала ленту карточек команд, которым кого-то не
хватает — свайпаешь и откликаешься
#4  [Аня, продакт]      ок
#5  [Миша, разработчик] Только у нас неделя до релиза, и я один на бэке
#6  [Дима, маркетолог]  Давайте сразу монетизацию — платное продвижение
анкеты
#7  [Аня, продакт]      Не, первые месяцы всё бесплатно, иначе никто не
придёт
#8  [Дима, маркетолог]  Без денег не окупим даже сервер
#9  [Аня, продакт]      Сервер тянем на гранте университета, монетизацию
обсудим после 500 пользователей
#10 [Миша, разработчик] Новости: Петя присоединился к бэку, и у нас две
недели
    → «Собрать план»
```

После	Ожидаем на канвасе	Агент в чате
#1	Суть про поиск сокомандников; Проблема «ищут в хаотичных чатах»; Для кого «студенты, хакатоны»	—
#3	**Одна цель:** в Решении один тезис про «быстро найти подходящую команду», дети — «анкеты и мэтчинг» [Миша] и «лента карточек со свайпом» [Лера]	Тост «Свёл 2 тезиса», в чат ничего
#4	Ничего не меняется (шум)	—
#5	Ограничение «неделя, 1 бэкендер». Допустимо: деталь агента «мэтчинг по тегам стека, без ML» с `depends_on` на это ограничение	Не больше одной фразы
#7	**Конфликт:** «платное продвижение» и «бесплатно первые месяцы» — `disputed`, красное ребро, вопрос в блоке вопросов. **Не слиты**	Называет конфликт (деньги сейчас vs рост аудитории) и спрашивает, что важнее
#8	Риск «не окупим сервер» или `attach_source` к тезису Димы — но не новый конфликт	—

После	Ожидаем на канвасе	Агент в чате
#9	Конфликт разрешён: «платное продвижение» → `rejected` с причиной «после 500 пользователей»; ограничение «сервер на гранте»; вопрос закрыт	—
#10	`change_constraint` на «неделя, 1 бэкендер» → «две недели, 2 бэкендера». Всё, что от него зависело, — `needs_review`	«Ограничение ослабло — можно вернуть полноценный мэтчинг?»
План	2–3 этапа: анкеты + лента карточек → мэтчинг. Все шаги со ссылками на узлы. Монетизации среди шагов нет	«План собран»

Чат 3. Предзаказ в кофейне у кампуса

Проверяет недопонимание, дубли и шум. Здесь агент должен больше молчать, чем говорить.

```
#1 [Оля, владелица]   Хочу, чтобы студенты заказывали кофе заранее и не
стояли в очереди между парами
#2 [Кирилл, разработчик] Нужен лендинг
#3 [Саша, дизайнер]   Нет, нужен не лендинг, а простая главная с меню и
кнопкой «заказать»
#4 [Кирилл, разработчик] так я про это и говорю)
#5 [Оля, владелица]   Очереди на большой перемене по 15 минут, люди
разворачиваются и уходят
#6 [Саша, дизайнер]   Да, очереди — главная боль, согласна
#7 [Кирилл, разработчик] ахах ок
#8 [Оля, владелица]   Оплату пока не делаем, платят на кассе при получении
```

После	Ожидаем на канвасе	Агент в чате
#1	Суть про предзаказ; Проблема «очереди между парами»; Для кого «студенты»	—
#3	Вариант А: два тезиса + вопрос «лендинг и простая главная — это одно и то же?» (`misunderstanding`). Вариант Б: сразу один тезис. Оба допустимы, **конфликт — нет**	Вариант А: один уточняющий вопрос

После	Ожидаем на канвасе	Агент в чате
#4	Один тезис «Простая главная: меню + кнопка заказа» с авторами Кирилл и Саша; вопрос, если был, закрыт	—
#5	Проблема уточнена или деталь «15 минут на большой перемене» — **не второй тезис про очереди**	—
#6	`attach_source`: «+1» на проблеме, нового узла нет	—
#7	Ничего не меняется	—
#8	Ограничение «без онлайн-оплаты, оплата на кассе»	—

Что проверяем во всех трёх

- [ ] Нет двух узлов с одинаковым смыслом
- [ ] Узлы из конфликта ни разу не сливались
- [ ] Шум не попадает на канвас
- [ ] У каждого узла есть `source_msgs`, клик подсвечивает правильные сообщения
- [ ] Агент пишет в чат только там, где это указано в таблице
- [ ] Все шаги плана ссылаются на узлы, отклонённое в план не попадает
- [ ] Тезисы не длиннее 10 слов, в блоке не больше 6 тезисов

Риски и страховка

Риск	Что делаем
На защите модель отвечает иначе, чем на репетиции	`CACHE_MODE=replay` + реплики через демо-панель, тексты совпадают до символа
Канвас «трясётся», тезисы прыгают	Фиксированный каркас, стабильные id, Consolidator не больше 6 операций за проход, `layout`-анимации
Конспект переврал сказанное	Клик по тезису подсвечивает исходные реплики; ручная правка + pin

Риск	Что делаем
Агент склеил конфликт	Guards в коде запрещают merge при `jointly_satisfiable = false` и для узлов открытого конфликта; тестовые чаты ловят регрессии
План содержит то, чего не обсуждали	Шаги без валидных `from_nodes` отбрасываются кодом
Агент тормозит	Индикатор статуса; Extractor на быстрой модели; Consolidator в фоне; replay-кэш
Нет интернета или Wi-Fi режет WebSocket	Бэк локально на одном ноутбуке, раздача с телефона; в крайнем случае все вкладки на одной машине
Фронт канваса не успевает	Рёбра конфликтов → красные бейджи «конфликт с N2» на карточках; без анимаций
Ничего не работает на защите	Запасное видео полного сценария, записанное в 17:00

Демо-панель (`?demo=1`) подставляет следующую реплику игрока из `demo_shop.json` в поле ввода, а Enter игрок жмёт сам. Та же панель умеет подставлять реплики из тестовых чатов — удобно для ручной проверки.